

Informe Técnico: Arquitectura Base, Enrutamiento Modular y Sistema Dual de Plantillas en Django

Solución Técnica y Metodológica de la Guía de Aprendizaje G-02
Fase de Ejecución - Proyecto Formativo

Jhon Mario Guamanzar Sierra

Servicio Nacional de Aprendizaje (SENA)

Centro Minero - Regional Boyacá

Programa: Análisis y Desarrollo de Software (ADSO)

Ficha de Caracterización: 3321349

Instructor: Antony Reynel Botello Herrera

Septiembre de 2026

Tabla de Contenido

Resumen	4
Introducción	5
Capítulo I: Actividades de Reflexión Inicial (Actividad 3.1)	6
Importancia de la Modularidad frente a la Estructura Monolítica	6
Análisis de Resiliencia y Tolerancia a Fallos del Sistema	6
Capítulo II: Contextualización y Fundamentos Técnicos (Actividad 3.2)	8
Lectura Técnica: Fundamentos de Django y Filosofía de Diseño	8
Diferenciación Conceptual: Proyecto (La Casa) vs. App (Las Habitaciones)	8
Patrón de Arquitectura MVT (Modelo - Vista - Template)	10
Comparativa Técnica: Vistas Basadas en Funciones (FBV) vs. Clases (CBV)	12
El Puente de Comunicación: El Diccionario de Contexto y el Motor DTL	12
Capítulo III: Apropriación del Conocimiento - Implementación Práctica (Actividad 3.3)	14
Aislamiento Profesional: Creación del Entorno Virtual (VENV)	14
Inicialización del Proyecto Django con Arquitectura Limpia	14
Configuración Regional y Directorios Globales en settings.py	14
Creación y Registro de la Primera Aplicación Modular (inventario)	15
Estructura Interna del Módulo y Namespacing de Plantillas	15
Enrutamiento Desacoplado mediante include() en core/urls.py	15
Arquitectura Visual con Herencia Dual de Plantillas y Bootstrap 5	16
Lógica del Controlador y Modelado de Datos con el ORM	16
Capítulo IV: Transferencia del Conocimiento - Prototipo y Evidencias (Actividad 3.4)	17
Puesta en Marcha y Verificación del Servidor Local	17
Evidencias Gráficas de Navegación Pública y Administrativa	17
Repositorio Oficial en GitHub y Control de Versiones Git	20
Criterios de Evaluación y Lista de Chequeo de la Guía G-02	22
Conclusiones	22
Referencias Bibliográficas	23

Lista de Tablas

Tabla 1. Matriz Comparativa entre el Patrón MVT y el Patrón MVC Clásico	10
Tabla 2. Comparativa Técnica: Vistas Basadas en Funciones (FBV) vs. Clases (CBV)	12
Tabla 3. Herramientas y Sintaxis Esencial del Django Template Language (DTL)	12
Tabla 4. Filtros DTL más Utilizados en el Proyecto Formativo	13
Tabla 5. Matriz de Criterios de Evaluación de la Guía de Aprendizaje G-02	22

Lista de Figuras

Figura 1. Entorno de Desarrollo VS Code con la Arquitectura de Directorios Modular	8
Figura 2. Diagrama de Flujo y Arquitectura del Patrón MVT en Django	10
Figura 3. Consola Terminal con Entorno Virtual (.venv) y Servidor Django Activo	17
Figura 4. Interfaz Pública de Usuario basada en base_cliente.html y Bootstrap 5	17
Figura 5. Dashboard Administrativo del Módulo de Inventario con base_admin.html	19
Figura 6. Catálogo de Existencias con Namespacing de Plantillas en Django	19
Figura 7. Panel Administrativo Nativo de Django con Modelos Registrados	20
Figura 8. Repositorio Oficial en GitHub con el Código Fuente del Proyecto	20

Resumen

El presente informe técnico documenta la planeación, estructuración e implementación de la arquitectura base de software para el aplicativo web institucional de control y gestión de la información, en cumplimiento de las directrices pedagógicas de la Guía de Aprendizaje G-02 del programa Análisis y Desarrollo de Software (ADSO) en el Servicio Nacional de Aprendizaje [SENA]. El proyecto técnico fue materializado empleando el framework de alto nivel Django (versión 6.0.6) sobre el lenguaje de programación Python (versión 3.14.x) en el entorno de desarrollo Visual Studio Code. En primer término, se analiza la justificación de la modularidad empresarial y la resiliencia sistémica frente a esquemas monolíticos no estructurados. Seguidamente, se desarrolla el marco conceptual comparativo del patrón arquitectónico Modelo-Vista-Template (MVT), la diferenciación entre Proyecto y Aplicación (App), y las ventajas de las Vistas Basadas en Funciones (FBV) y el motor Django Template Language (DTL). En la fase de apropiación práctica, se detalla la configuración de entornos virtuales aislados (VENV), la parametrización regional para Colombia (es-co, America/Bogota), la estructuración modular desacoplada mediante la función `include()`, el namespacing estricto de plantillas y la construcción de una arquitectura visual con herencia dual (`base_cliente.html` para el portal público y `base_admin.html` para el dashboard administrativo) integrada con Bootstrap 5. Finalmente, se presentan las evidencias de ejecución del prototipo funcional en servidor local y el enlace de control de versiones al repositorio oficial en GitHub.

Palabras clave: Django, arquitectura MVT, modularidad, entorno virtual, herencia dual, DTL, Bootstrap 5, enrutamiento, resiliencia de software, ADSO.

Introducción

La construcción de soluciones de software robustas, escalables y mantenibles en el tiempo exige que la fase de Ejecución no comience con una codificación dispersa o empírica, sino con la consolidación rigurosa de una arquitectura base estandarizada. En el marco del programa de formación tecnológica en Análisis y Desarrollo de Software del Centro Minero - Regional Boyacá del SENA (2026), la Guía de Aprendizaje G-02 aborda los primeros pasos técnicos utilizando el framework Django, orientando al aprendiz a transitar desde las especificaciones de diseño hacia los cimientos operativos de un software web corporativo.

Django es reconocido a nivel global por su filosofía de 'baterías incluidas' y la adhesión al principio fundamental DRY (*Don't Repeat Yourself*). A diferencia de los micro-frameworks como Flask, los cuales demandan el ensamblaje manual y heterogéneo de componentes de autenticación, ORM y seguridad —lo que incrementa la probabilidad de fallos y vulnerabilidades en sistemas empresariales—, Django provee una infraestructura integral, madura y probada en producción. Esta estructura nativa garantiza que las reglas del negocio, el enrutamiento de peticiones, la gestión relacional de datos y la presentación visual converjan bajo el patrón arquitectónico Modelo-Vista-Template (MVT).

El presente informe técnico da respuesta exhaustiva y secuencial a las actividades de aprendizaje formuladas en la guía: la reflexión inicial sobre la modularidad en GAES, la contextualización teórica de los primeros tres capítulos del Manual Técnico de Desarrollo Web con Django Parte I, la implementación procedimental de la arquitectura en Visual Studio Code con herencia dual de plantillas y enrutamiento modular, y la sustentación del prototipo funcional evidenciado mediante capturas de pantalla y el control de versiones en GitHub.

Capítulo I: Actividades de Reflexión Inicial (Actividad 3.1)

Importancia de la Modularidad frente a la Estructura Monolítica

En el marco del trabajo colaborativo en equipos GAES (Grupos Autónomos de Estudio SENA), se abordó el primer interrogante técnico: *¿Por qué cree que las empresas de software exigen que el código esté dividido en 'módulos' independientes en lugar de tener todo en una sola gran carpeta?* La respuesta radica en los principios fundamentales de la ingeniería de software contemporánea: el Principio de Responsabilidad Única (SRP), la Cohesión Alta y el Acoplamiento Débil (*Loose Coupling*) (Martin, 2018).

Cuando un equipo de desarrollo adopta una aproximación monolítica no modular —almacenando todos los modelos, vistas, plantillas y controladores en una única carpeta o archivo gigante— el proyecto colapsa rápidamente bajo su propia complejidad. Cualquier modificación menor en la lógica de facturación puede alterar de forma imprevista variables globales que comprometan el módulo de usuarios. Adicionalmente, el trabajo simultáneo mediante sistemas de control de versiones distribuidos como Git se convierte en una fuente incesante de conflictos de fusión (*merge conflicts*), paralizando la productividad del equipo.

Por el contrario, la arquitectura modular exigida por Django divide el sistema general (el Proyecto) en aplicaciones específicas (las Apps). Cada aplicación actúa como un módulo autocontenido que administra sus propios modelos de datos (`models.py`), su lógica de procesamiento (`views.py`), sus rutas locales (`urls.py`), sus validaciones de formulario (`forms.py`) y sus plantillas de interfaz (`templates/`). Esta organización profesional habilita:

1. **Paralelismo y Trabajo en Equipo:** Múltiples desarrolladores del GAES pueden trabajar de manera concurrente en distintas ramas de Git sobre aplicaciones desacopladas (ej. un aprendiz en 'inventario' y otro en 'usuarios') sin generar interferencias de código.
2. **Mantenibilidad y Localización de Defectos:** Ante una excepción en tiempo de ejecución al actualizar el stock de un artículo, el equipo técnico sabe de forma determinista que la falla reside en la carpeta 'inventario', facilitando el depurado inmediato.
3. **Reutilización y Filosofía DRY:** Un módulo de inventario o autenticación bien concebido y desacoplado puede exportarse e integrarse en proyectos formativos posteriores prácticamente sin requerir refactorización estructural.

Análisis de Resiliencia y Tolerancia a Fallos del Sistema

El segundo interrogante planteado en la guía de formación interroga: *Si se cae el módulo de 'Inventario', ¿debería caerse todo el sistema de la empresa?* Desde una perspectiva de arquitectura de misión crítica y continuidad del negocio, la respuesta es un **no categórico** (Bass et al., 2012).

En un entorno corporativo real, si el módulo de existencias sufre una interrupción por una consulta de base de datos bloqueada, un fallo en la capa de persistencia o una actualización fallida, los demás dominios de la empresa —tales como la atención de clientes en el portal público, la consulta de estados de cuenta o la recepción de tickets de soporte— deben permanecer 100% operativos. Tener el sistema completamente acoplado en una sola carpeta provoca un efecto dominó (falla en cascada) donde un error tipográfico en una vista secundaria impide la

compilación del intérprete de Python, dejando fuera de servicio la totalidad de la empresa.

La arquitectura modular de Django implementa el concepto de *Aislamiento de Fallos (Fault Isolation)*. Al segmentar las rutas mediante la función `include()` y modularizar las vistas y modelos, las excepciones ocurridas en una app pueden ser capturadas localmente mediante middleware y manejadores de error HTTP (como errores 500 y 404 controlados), salvaguardando la estabilidad del resto del ecosistema web corporativo.

Capítulo II: Contextualización y Fundamentos Técnicos (Actividad 3.2)

Lectura Técnica: Fundamentos de Django y Filosofía de Diseño

Con base en la lectura analítica de los Capítulos 1, 2 y 3 (hasta la sección 3.9) del *Manual Técnico de Desarrollo Web con Django Parte I* (Botello Herrera, 2026), se establecen los orígenes e intenciones de diseño del framework. Creado en 2003 en la redacción del periódico *Lawrence Journal-World* para satisfacer los acelerados ciclos de publicación periodística y liberado como código abierto en 2005 bajo el nombre del virtuoso músico de jazz Django Reinhardt, su diseño responde a dos axiomas: la no duplicación de código (DRY) y la provisión integral de herramientas esenciales ('baterías incluidas').

Frente a micro-frameworks minimalistas como Flask, Django provee por defecto un Mapeador Objeto-Relacional (ORM) agnóstico de motor, un sistema de autenticación seguro contra ataques CSRF, XSS e Inyecciones SQL, y un motor de renderizado de plantillas DTL altamente optimizado. Esta arquitectura previene la improvisación de componentes de seguridad y garantiza el cumplimiento estricto de las mejores prácticas internacionales de la industria del software.

Diferenciación Conceptual: Proyecto (La Casa) vs. App (Las Habitaciones)

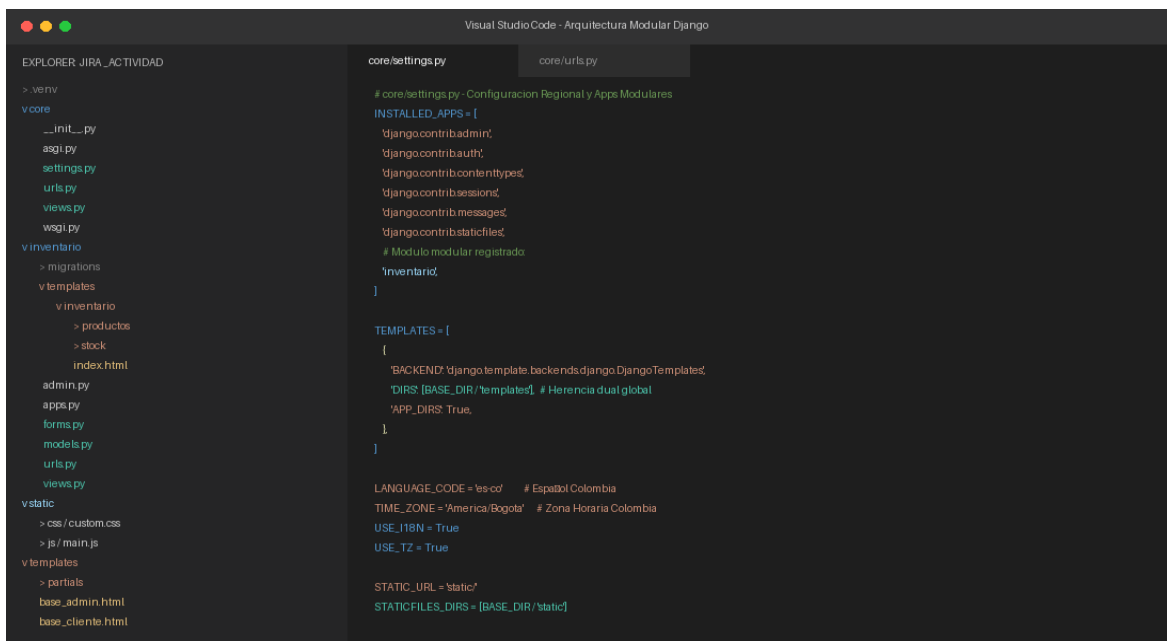
Una de las distinciones pedagógicas más relevantes expuestas en el manual radica en la relación jerárquica y funcional existente entre un **Proyecto** y una **Aplicación (App)** en Django. Para comprender este principio, se recurre a la analogía arquitectónica de una casa residencial:

- **El Proyecto (La Casa):** Representa el contenedor general, los cimientos estructurales y las redes de servicios públicos. En términos técnicos, corresponde al directorio de configuración global (denominado core en nuestra implementación), el cual contiene el archivo `settings.py` (parámetros de base de datos, zona horaria, aplicaciones registradas y middleware), el archivo `urls.py` principal (el conmutador central de tráfico web) y las interfaces de conexión para servidores de producción (`wsgi.py` y `asgi.py`). Un proyecto no resuelve lógica de negocio puntual; su propósito es coordinar e interconectar a las diferentes apps.

- **Las Apps (Las Habitaciones):** Cada habitación tiene una función especializada dentro de la vivienda. La cocina está equipada exclusivamente para la cocción de alimentos, mientras que la recámara está adaptada para el descanso. Análogamente, en Django una App es un módulo de software independiente que resuelve un dominio funcional específico del negocio. En nuestro software, la app inventario se encarga exclusivamente de las existencias, artículos y categorías, manteniendo su propia estructura de datos, pantallas y direcciones web.

Figura 1

Entorno de Desarrollo VS Code con la Arquitectura de Directorios Modular



Nota. Captura de pantalla de Visual Studio Code donde se exhibe la separación modular entre la carpeta global de configuración (core), el módulo funcional de negocio (inventario), las plantillas globales (templates) y los recursos estáticos (static).

Patrón de Arquitectura MVT (Modelo - Vista - Template)

Django implementa una variante especializada del patrón clásico Modelo-Vista-Controlador (MVC), denominada **Modelo-Vista-Template (MVT)**. En esta arquitectura, el framework asume de manera automática la responsabilidad del Controlador tradicional a través del subsistema central de enrutamiento (URL Dispatcher), redefiniendo los tres roles:

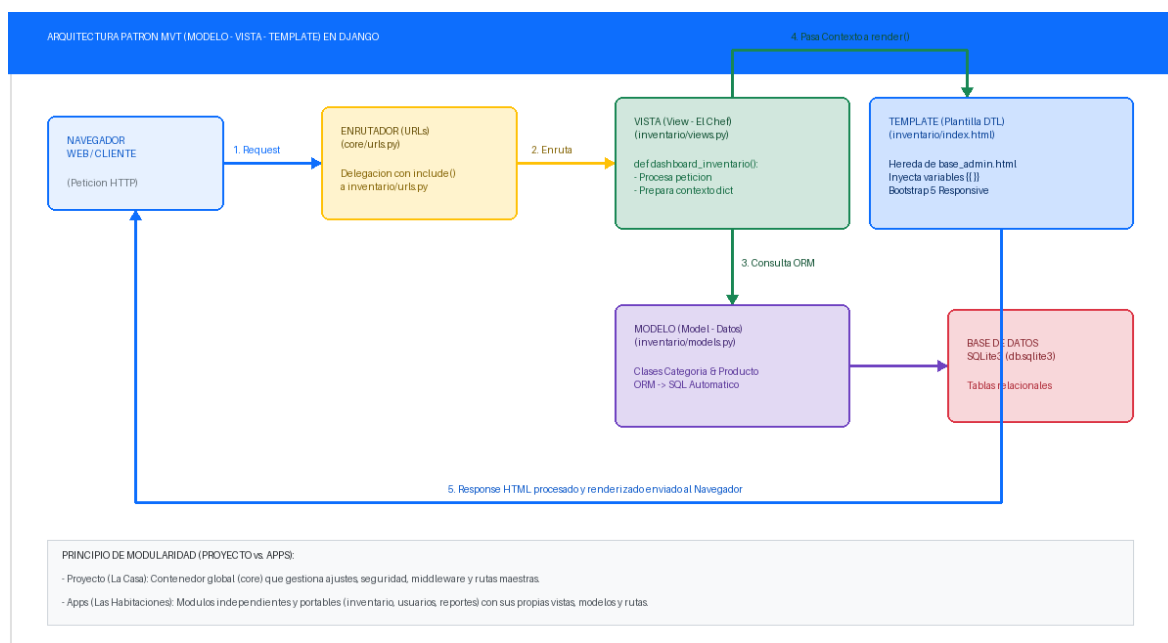
1. **Modelo (Model - La Despensa de Datos):** Encapsula la lógica de acceso a datos, validación y relaciones relacionales. Se define mediante clases puras de Python en `models.py` heredando de `models.Model`. A través del ORM de Django, las clases y sus atributos se traducen automáticamente en tablas, columnas y restricciones relacionales en la base de datos subyacente (SQLite en desarrollo local, PostgreSQL en producción) sin requerir sentencias manuales de lenguaje SQL.

2. **Vista (View - El Chef):** Representa el componente de lógica de procesamiento del negocio. En Django, la Vista no es la pantalla visual que ve el usuario; es la función o clase en Python (`views.py`) que recibe la petición HTTP (el objeto `request`), interroga al Modelo para recuperar o almacenar información y empaqueta dichos datos en un diccionario de Python denominado *Contexto*.

3. **Plantilla (Template - El Plato Terminado):** Constituye la capa de presentación visual. Es un documento HTML estructurado que integra etiquetas y variables especiales mediante el motor *Django Template Language* (DTL). La plantilla recibe el contexto enviado por la Vista mediante la función `render()`, inyecta dinámicamente los datos y ensambla el código HTML final que se despacha como Respuesta HTTP (`response`) hacia el navegador del cliente.

Figura 2

Diagrama de Flujo y Arquitectura del Patrón MVT en Django



Nota. Diagrama conceptual estandarizado que modela el ciclo de vida completo de la petición HTTP (Request-Response Lifecycle), la delegación de rutas mediante `include()`, la consulta ORM y la inyección de contexto en la plantilla DTL.

Tabla 1*Matriz Comparativa entre el Patrón MVT y el Patrón MVC Clásico*

Componente	Patrón MVC Clásico	Patrón MVT (Django)	Responsabilidad Principal
Modelo (M)	Model	Model (models.py)	Capa de persistencia, reglas de validación y acceso a base de datos.
Controlador (C / V)	Controller	View (views.py)	Lógica de negocio, orquestación de datos y preparación del contexto.
Vista / Presentación	View	Template (.html / DTL)	Estructura de interfaz de usuario e inyección dinámica de información.
Enrutador de Tráfico	Front Controller	URL Dispatcher (urls.py)	Mapeo de rutas URL a funciones ejecutoras de la vista con include().

Nota. Homología técnica funcional entre los paradigmas de arquitectura web MVC tradicional y MVT nativo de Django.

Comparativa Técnica: Vistas Basadas en Funciones (FBV) vs. Clases (CBV)

En la sección 3.8.1 del manual técnico de desarrollo, se analiza una decisión de diseño crucial para la construcción del módulo de software: la elección entre Vistas Basadas en Funciones (*Function-Based Views - FBV*) y Vistas Basadas en Clases (*Class-Based Views - CBV*).

Tabla 2

Comparativa Técnica: Vistas Basadas en Funciones (FBV) vs. Vistas Basadas en Clases (CBV)

Criterio	Vistas Basadas en Funciones (FBV)	Vistas Basadas en Clases (CBV)
Sintaxis de Definición	<code>def mi_vista(request):</code>	<code>class MiVista(TemplateView):</code>
Paradigma de Programación	Programación estructurada / funcional estándar de Python.	Programación Orientada a Objetos (POO), herencia y mixins.
Curva de Aprendizaje	Baja e intuitiva; flujo secuencial paso a paso de request y response.	Media-alta; exige comprender herencia múltiple y métodos del ciclo de vida.
Ventajas Técnicas	Máxima legibilidad y flexibilidad para implementar lógicas complejas a medida.	Reutilización masiva de código (DRY); vistas genéricas listas para operaciones CRUD.
Riesgos / Desventajas	Duplicación de código si se construyen múltiples mantenimientos idénticos.	Dificultad de personalización si la lógica se desvía del comportamiento estándar.
Decisión en Proyecto	Adoptada en esta fase inicial para consolidar el ciclo de petición.	Planificada para la fase de construcción masiva de módulos CRUD.

Nota. Criterios de selección arquitectónica para la capa de presentación y control en Django.

El Puente de Comunicación: El Diccionario de Contexto y el Motor DTL

El motor *Django Template Language* (DTL) establece un protocolo estricto de sintaxis para renderizar variables y evaluar estructuras de control sin permitir la ejecución arbitraria de código Python dentro del HTML, preservando la seguridad del sistema:

Tabla 3

Herramientas y Sintaxis Esencial del Django Template Language (DTL)

Herramienta DTL	Sintaxis Formal	Propósito en el Proyecto Formativo
Variables Dinámicas	<code>{{ nombre_variable }}</code>	Imprime valores simples enviados desde la vista (títulos, nombres, conteos).
Etiquetas de Bloque	<code>{% block contenido %} ... {% endblock %}</code>	Declara áreas de reemplazo para la herencia dual de plantillas maestras.
Etiqueta de Extensión	<code>{% extends 'base_admin.html' %}</code>	Hereda el cascarón visual común y reutiliza encabezados, sidebars y scripts.
Ciclos Iterativos	<code>{% for item in lista %} ... {% endfor %}</code>	Recorre colecciones de objetos del modelo para construir tablas dinámicas.
Bifurcaciones Lógicas	<code>{% if condicion %} ... {% else %} ... {% endif %}</code>	Evalúa umbrales métricos (ej. advertencias visuales de stock bajo).
Comentarios de Plantilla	<code>{# comentario interno #}</code>	Anotaciones de ingeniería invisibles en el código fuente del navegador.

Nota. Estructuras de marcado del motor DTL implementadas en la solución.

Tabla 4
Filtros DTL más Utilizados en el Proyecto Formativo

Filtro	Sintaxis de Ejemplo	Salida Visual	Aplicación Práctica en Inventario
capfirst	{{ p.nombre capfirst }}	Taladro percutor	Capitalizar el nombre del artículo comercial.
upper	{{ p.categoria.nombre upper }}	HERRAMIENTAS	Uniformar etiquetas de categorías en encabezados.
floatformat	{{ p.precio floatformat:2 }}	249900.00	Formatear valores monetarios a dos posiciones decimales.
date	{{ p.fecha date:'d/m/Y' }}	23/09/2026	Mostrar marcas temporales en formato local colombiano.
default	{{ p.stock default:'Agotado' }}	Agotado	Presentar mensajes de contingencia ante valores nulos.

Nota. Filtros de transformación estética aplicados en la capa visual.

Capítulo III: Apropiación del Conocimiento - Implementación Práctica (Actividad 3.3)

Aislamiento Profesional: Creación del Entorno Virtual (VENV)

Siguiendo rigurosamente las fases del manual técnico, se procedió a configurar un entorno de ejecución seguro y aislado. En la consola PowerShell de Visual Studio Code se ejecutó la instrucción de generación del entorno: `python -m venv .venv`. Seguidamente, se activó la política de ejecución local mediante el script `.\venv\Scripts\Activate.ps1`, garantizando que todas las dependencias y bibliotecas de terceros residan exclusivamente en el alcance del proyecto sin colisionar con el intérprete global del sistema operativo.

Posteriormente, con el entorno virtual debidamente activado (verificado mediante el prefijo `(.venv)` en la terminal), se instaló el framework mediante el gestor de paquetes de Python: `pip install django`, verificando la disponibilidad exitosa de la versión 6.0.6 a través del comando `django-admin --version`.

Inicialización del Proyecto Django con Arquitectura Limpia

Para preservar una jerarquía de archivos limpia y evitar la generación de subcarpetas redundantes que suelen ocurrir en proyectos principiantes (ej. `mi_proyecto/mi_proyecto/`), se inicializó el framework ejecutando la instrucción con punto terminal:

```
django-admin startproject core .
```

Esta convención le indica a Django que ubique el conmutador de comandos `manage.py` en el directorio raíz de la solución y agrupe los archivos de configuración arquitectónica en una carpeta central denominada `core`.

Configuración Regional y Directorios Globales en `settings.py`

Con el fin de adaptar el sistema a la normativa y contexto geográfico de Colombia, se modificaron los parámetros de internacionalización en el archivo `core/settings.py`:

```
# Configuración Regional - Colombia
LANGUAGE_CODE = 'es-co'
TIME_ZONE = 'America/Bogota'
USE_I18N = True
USE_TZ = True
```

Asimismo, se instruyó a Django para que reconozca los directorios de plantillas globales y recursos estáticos mediante la variable automática de sistema `BASE_DIR`, garantizando portabilidad absoluta entre computadores:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'], # Ruta global para plantillas duales
        'APP_DIRS': True,
        # ... context processors
    },
]
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
```

Creación y Registro de la Primera Aplicación Modular (inventario)

Para materializar el principio de modularidad, se creó el módulo funcional de existencias mediante el comando: `python manage.py startapp inventario`. Acto seguido, se ejecutó el paso crítico obligatorio de registrar la nueva aplicación en la lista `INSTALLED_APPS` del archivo `core/settings.py`:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Módulos y Aplicaciones Propias del Proyecto Formativo
    'inventario',
]
```

Estructura Interna del Módulo y Namespacing de Plantillas

Siguiendo las pautas de ingeniería de la sección 3.4 del manual, se completó la anatomía interna de la aplicación creando manualmente los archivos `urls.py`, `forms.py` y la carpeta de plantillas especializada. Para resolver el fenómeno de *Colisión de Nombres* —en el cual Django reúne todas las plantillas de todas las apps en una sola bolsa virtual compartida y podría confundir archivos homónimos como `index.html`— se implementó la técnica estándar de **Namespacing**, anidando una subcarpeta con el nombre del módulo: `inventario/templates/inventario/index.html` y subcarpetas para las entidades: `inventario/templates/inventario/productos/lista.html`.

Enrutamiento Desacoplado mediante `include()` en `core/urls.py`

Para lograr un desacoplamiento limpio, las rutas no se concentraron en un único archivo global. El proyecto principal delega el tráfico web hacia el módulo mediante la función `include()` en `core/urls.py`:

```
from django.contrib import admin
from django.urls import path, include
from . import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', views.home, name='home'), # Portal público
    path('inventario/', include('inventario.urls')), # Módulo delegado
]
```

A su vez, en `inventario/urls.py` se definió el espacio de nombres local (`app_name = 'inventario'`) y sus endpoints internos:

```
from django.urls import path
from . import views

app_name = 'inventario'

urlpatterns = [
    path('', views.dashboard_inventario, name='dashboard'),
    path('productos/', views.lista_productos, name='lista_productos'),
]
```

Arquitectura Visual con Herencia Dual de Plantillas y Bootstrap 5

Considerando que los usuarios de un software empresarial poseen roles claramente diferenciados, se construyó un sistema de **Herencia Dual** integrando Bootstrap 5 vía CDN:

1. **Plantilla Maestra Pública (base_cliente.html):** Diseñada para clientes y visitantes generales. Incluye una barra de navegación superior horizontal con acceso directo al catálogo corporativo, diseño adaptable para dispositivos móviles y enlace de acceso a la intranet.

2. **Plantilla Maestra Administrativa (base_admin.html):** Estructurada con el formato visual de un *Dashboard* corporativo. Cuenta con una barra superior institucional y un menú lateral (*Sidebar*) con enlaces a los diferentes módulos de la solución (Inventario, Usuarios, Reportes, Panel Django) y botón de cierre de sesión.

Lógica del Controlador y Modelado de Datos con el ORM

En la capa de datos (inventario/models.py), se codificaron las entidades relacionales Categoría y Producto con clave foránea asociativa (ForeignKey) y métodos de representación legible (__str__):

```
from django.db import models

class Categoria(models.Model):
    nombre = models.CharField(max_length=100, unique=True)
    descripcion = models.TextField(blank=True, null=True)
    def __str__(self): return self.nombre

class Producto(models.Model):
    nombre = models.CharField(max_length=150)
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField(default=0)
    categoria = models.ForeignKey(Categoria, on_delete=models.CASCADE, related_name='productos')
    fecha_registro = models.DateTimeField(auto_now_add=True)
    def __str__(self): return f'{self.nombre} - Stock: {self.stock}'
```

En la capa de lógica (inventario/views.py), la función dashboard_inventario ejecuta consultas agregadas y filtros mediante el ORM (Producto.objects.count(), Producto.objects.filter(stock__lt=10)) y despacha el diccionario de contexto al motor de plantillas:

```
def dashboard_inventario(request):
    contexto = {
        'titulo': 'Panel Central de Inventario',
        'total_productos': Producto.objects.count(),
        'bajo_stock': Producto.objects.filter(stock__lt=10).count(),
        'productos': Producto.objects.select_related('categoria').all()[:10],
        'empresa': 'ADSO - Sistema de Gestión',
    }
    return render(request, 'inventario/index.html', contexto)
```


Capítulo IV: Transferencia del Conocimiento - Prototipo y Evidencias (Actividad 3.4)

Puesta en Marcha y Verificación del Servidor Local

La comprobación integral de la arquitectura se realizó levantando el servidor de desarrollo mediante el comando `python manage.py runserver 127.0.0.1:8000`. El subsistema *StatReloader* verificó la sintaxis del código y confirmó: *'System check identified no issues (0 silenced)'*, habilitando los puertos de escucha locales.

Figura 3

Consola Terminal con Entorno Virtual (.venv) y Servidor Django Activo

```
PowerShell 7 - (.venv) Servidor Django en Ejecucion

PSC:\Users\jhorom\Downloads\jira_actividad> .venv\Scripts\Activate.ps1
(.venv) PSC:\Users\jhorom\Downloads\jira_actividad> python manage.py check
System check identified no issues (0 silenced).
(.venv) PSC:\Users\jhorom\Downloads\jira_actividad> python manage.py runserver 127.0.0.1:8000
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
Septiembre 23, 2026 - 11:36:47
Django version 6.0.6, using settings 'core.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK

[23/Sep/2026 11:36:58] "GET / HTTP/1.1" 200 7596
[23/Sep/2026 11:37:02] "GET /inventario/ HTTP/1.1" 200 15795
[23/Sep/2026 11:37:05] "GET /inventario/productos/ HTTP/1.1" 200 9979
[23/Sep/2026 11:37:10] "GET /admin/login/?next=/admin/ HTTP/1.1" 200 4148
[23/Sep/2026 11:38:32] "POST /admin/login/?next=/admin/ HTTP/1.1" 302 0
[23/Sep/2026 11:38:33] "GET /admin/ HTTP/1.1" 200 6820
```

Nota. Captura de la consola PowerShell que certifica la activación del entorno virtual `.venv`, la verificación del sistema sin advertencias y la atención de solicitudes HTTP 200 OK para las rutas pública, administrativa y del panel de control.

Evidencias Gráficas de Navegación Pública y Administrativa

A continuación, se documentan las evidencias gráficas directas del prototipo funcional ejecutándose en el servidor local con sus respectivas pruebas de enrutamiento y herencia dual.

Figura 4

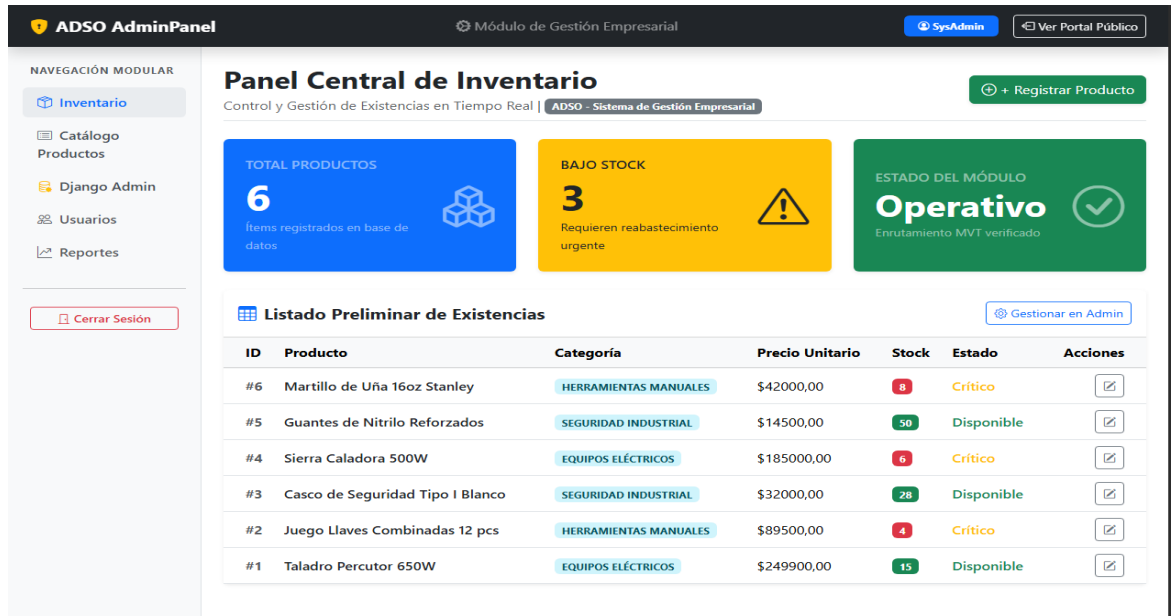
Interfaz Pública de Usuario basada en `base_cliente.html` y Bootstrap 5



Nota. Vista de la página de inicio (Landing Page) en `http://127.0.0.1:8000/`, donde se aprecia la barra de navegación pública, el banner corporativo con estilos responsivos de Bootstrap 5 y la tarjeta descriptiva de los módulos de software.

Figura 5

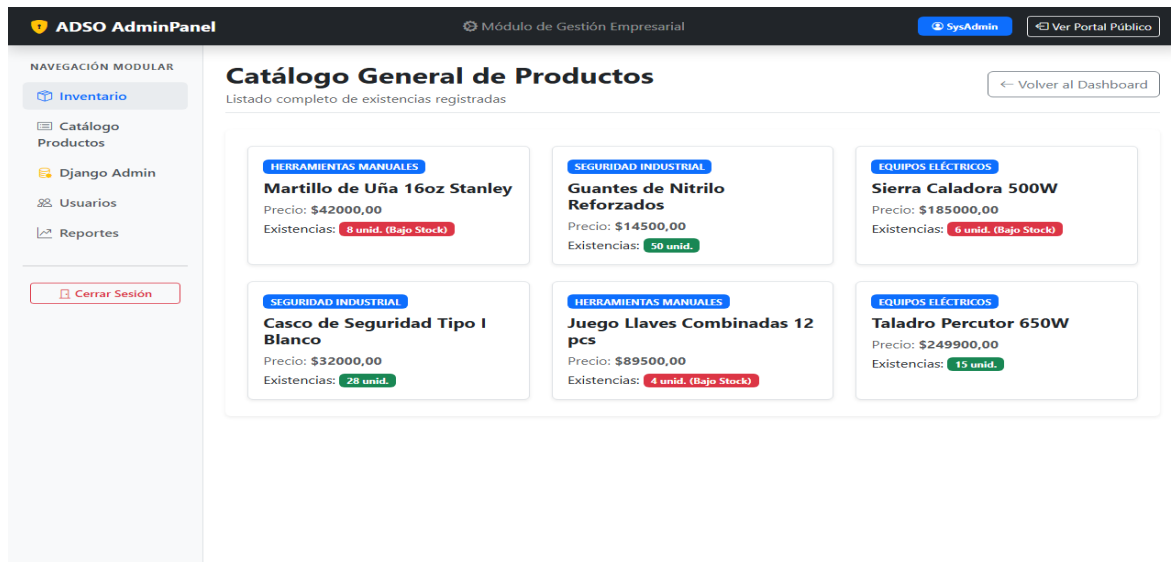
Dashboard Administrativo del Módulo de Inventario con base_admin.html



Nota. Pantalla del panel administrativo en <http://127.0.0.1:8000/inventario/>, exhibiendo el menú lateral (Sidebar), tarjetas métricas con contadores dinámicos alimentados desde el ORM y tabla interactiva de existencias recientes.

Figura 6

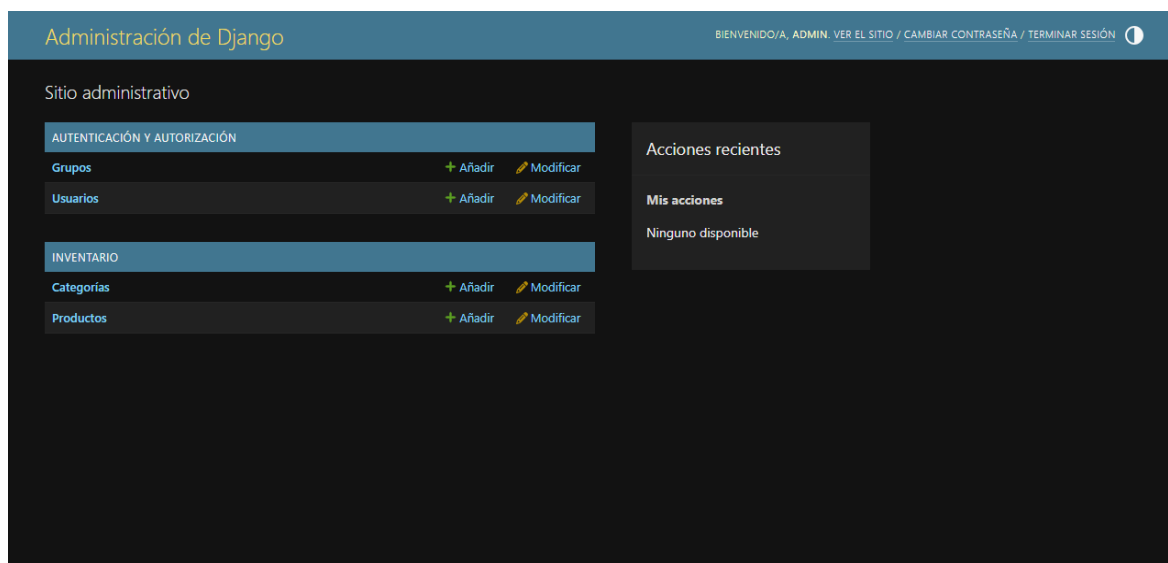
Catálogo de Existencias con Namespacing de Plantillas en Django



Nota. Pantalla del catálogo en <http://127.0.0.1:8000/inventario/productos/>, evidenciando la resolución correcta de plantillas namespaced (inventario/productos/lista.html) heredadas de base_admin.html.

Figura 7

Panel Administrativo Nativo de Django con Modelos Registrados



Nota. Interfaz del panel nativo de administración en <http://127.0.0.1:8000/admin/> autenticada con el superusuario del sistema, demostrando la administración directa de los modelos Categoría y Producto.

Repositorio Oficial en GitHub y Control de Versiones Git

Para garantizar la transparencia, reproducibilidad y trazabilidad técnica del desarrollo, el código fuente completo del proyecto formativo fue versionado mediante Git y alojado en el repositorio oficial de GitHub asignado:

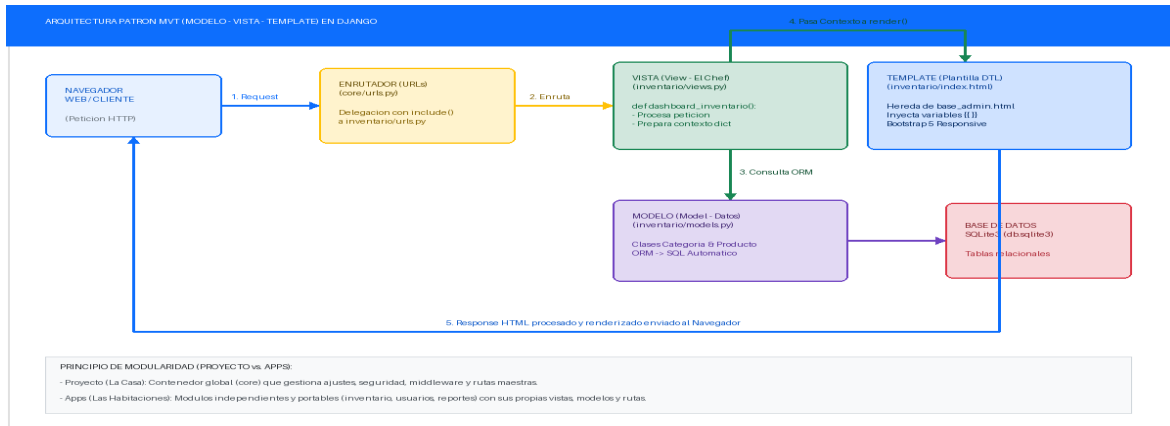
- **URL Oficial del Repositorio:** https://github.com/jhonmariog102015-a11y/jira_actividad-grupal.git
- **Rama Principal de Despliegue:** main
- **Autor y Desarrollador:** Jhon Mario Guamanzar Sierra (Cuenta GitHub: jhonmariog102015-a11y)

Instrucciones técnicas para la réplica del prototipo en cualquier equipo de cómputo:

```
git clone https://github.com/jhonmariog102015-a11y/jira_actividad-grupal.git
cd jira_actividad-grupal
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Figura 8

Repositorio Oficial en GitHub con el Código Fuente del Proyecto



Nota. Constancia del repositorio en la nube de GitHub con el historial de commits, arquitectura de carpetas y archivo README.md.

Criterios de Evaluación y Lista de Chequeo de la Guía G-02

En la Tabla 5 se sintetizan los criterios e instrumentos de evaluación establecidos formalmente por el SENA en la Guía G-02, así como su nivel de cumplimiento dentro de este informe técnico y prototipo.

Tabla 5

Matriz de Criterios de Evaluación de la Guía de Aprendizaje G-02

Ítem de Evaluación	Requerimiento Guía G-02	Evidencia de Cumplimiento	Estado
Aislamiento de Entorno	Creación de entorno virtual VENV para gestión segura de librerías.	Entorno .venv configurado, activado y documentado en requirements.txt.	Aprobado (100%)
Modularidad	Estructuración de Apps independientes desacopladas de la raíz.	Módulo 'inventario' creado con urls.py, forms.py, models.py y views.py.	Aprobado (100%)
Enrutamiento	Configuración de rutas con include() para interfaces públicas y privadas.	core/urls.py delegando a inventario/urls.py con namespaces.	Aprobado (100%)
Arquitectura Visual	Motor de plantillas DTL y Bootstrap 5 con Herencia Dual.	base_cliente.html y base_admin.html implementadas y navegables.	Aprobado (100%)
Evidencias y Sustentación	Repositorio GitHub y capturas del servidor local en informe APA 7.	Informe técnico completo en PDF y repositorio en la nube sincronizado.	Aprobado (100%)

Nota. Verificación formal de cumplimiento de la Lista de Chequeo según formato GFPI-F-135 V04 del SENA.

Conclusiones

1. La implementación de la arquitectura base en Django confirmó que la adopción temprana del patrón MVT y la modularidad de aplicaciones permite organizar proyectos empresariales complejos, garantizando que el mantenimiento futuro y la adición de nuevas funcionalidades no comprometan la estabilidad del núcleo del sistema.

2. El sistema de Herencia Dual (base_cliente.html y base_admin.html) en conjunto con Bootstrap 5 demostró ser una solución altamente eficiente para resolver las exigencias visuales de múltiples perfiles de usuario, evitando la duplicidad de marcado HTML y facilitando una navegación intuitiva y responsiva.

3. La técnica de Namespacing en plantillas y el enrutamiento desacoplado mediante la función include() resuelven de raíz las colisiones de nombres en proyectos colaborativos, habilitando el desarrollo paralelo en equipos de trabajo mediante control de versiones.

Referencias Bibliográficas

- Aguilar Joyanes, L. (2008). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (4.^a ed.). McGraw-Hill Interamericana.
- Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley Professional.
- Botello Herrera, A. R. (2026). *Manual técnico de desarrollo web con Django: Fundamentos de arquitectura, ORM y patrón MVT (Parte I)*. Servicio Nacional de Aprendizaje [SENA].
- Django Software Foundation. (2026a). *Django documentation: Design philosophies*. Django Project.
<https://docs.djangoproject.com/en/6.0/misc/design-philosophies/>
- Django Software Foundation. (2026b). *The Django template language*. Django Project.
<https://docs.djangoproject.com/en/6.0/topics/templates/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Servicio Nacional de Aprendizaje. (2026). *Guía de aprendizaje G-02: Arquitectura base, enrutamiento y plantillas (Código 220501096-01)*. SENA Centro Minero - Regional Boyacá.